UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

# Worksheet 7

**Student Name: Aradhya Sharma** **UID: 25MCC20042**
**Branch: MCA (Cloud Computing & Devops)** **Section/Group: MCC-101 A**
**Semester: 2** **Date of Performance: 31-03-2026**
**Subject Name: Technical Training -I** **Subject Code: 25CAP-652**

**Aim**

Implementation of joins in PostgreSQL (inner join ,left join,right join, multiple join and cross join)

**Tools Used**

- PostgreSQL
- pgAdmin
- Windows Operating System

**Objectives**

Apply joins to a real-world database schema (e.g., Students, Courses, Enrollments, Departments)

## Theory

A **JOIN** in SQL is used to combine rows from two or more tables based on a related column between them. Joins are essential in relational databases because data is often stored in multiple tables to reduce redundancy and improve organization.

**Types of Joins Used in this Experiment:**

1. **INNER JOIN:**
   Returns only the matching records from both tables.

2. **LEFT JOIN:**
   Returns all records from the left table and the matching records from the right table. If no match is found, NULL values are returned for the right table columns.
3. **RIGHT JOIN:**
   Returns all records from the right table and the matching records from the left table. If no match is found, NULL values are returned for the left table columns.
4. **MULTIPLE JOIN:**
   A multiple join in SQL means joining more than two tables in a single query to combine related data.
5. **CROSS JOIN:**
   Returns the Cartesian product of two tables, i.e., all possible combinations of rows from both tables.

## Experiment Steps

### Step 0: Creating tables and inserting records

## Code

```
CREATE TABLE Departments (

   dept_id INT PRIMARY KEY,

   dept_name VARCHAR(100)

);


CREATE TABLE Students (

   student_id INT PRIMARY KEY,

   student_name VARCHAR(100),

   dept_id INT,

   FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)

);


CREATE TABLE Courses (

   course_id INT PRIMARY KEY,

   course_name VARCHAR(100)

);
```

```
CREATE TABLE Enrollment (
   student_id INT,
   course_id INT,
   PRIMARY KEY (student_id, course_id),
   FOREIGN KEY (student_id) REFERENCES Students(student_id),
   FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

**Students Table :**

| | student_id [PK] integer | student_name character varying (100) | dept_id integer |
|---|---|---|---|
| 1 | 101 | Aradhya | 1 |
| 2 | 102 | Rohan | 2 |
| 3 | 103 | Priya | 1 |
| 4 | 104 | Amit | 3 |
| 5 | 105 | Neha | [null] |

**Courses Table:**

| | course_id [PK] integer | course_name character varying (100) |
|---|---|---|
| 1 | 201 | DBMS |
| 2 | 202 | Operating Systems |
| 3 | 203 | Thermodynamics |
| 4 | 204 | Networks |

**Enrollment Table:**

| | student_id [PK] integer | course_id [PK] integer |
|---|---|---|
| 1 | 101 | 201 |
| 2 | 101 | 202 |
| 3 | 102 | 203 |
| 4 | 103 | 201 |

**Departments Table :**

| | dept_id [PK] integer | dept_name character varying (100) |
|---|---|---|
| 1 | 1 | Computer Science |
| 2 | 2 | Mechanical |
| 3 | 3 | Electrical |

## Step 1 : Queries to list students with their enrolled courses (INNER JOIN).

```
SELECT s.student_id, s.student_name, c.course_name

FROM Students s

INNER JOIN Enrollment e ON s.student_id = e.student_id

INNER JOIN Courses c ON e.course_id = c.course_id;
```

## Output:

| | student_id integer | student_name character varying (100) | course_name character varying (100) |
|---|---|---|---|
| 1 | 101 | Aradhya | DBMS |
| 2 | 101 | Aradhya | Operating Systems |
| 3 | 102 | Rohan | Thermodynamics |
| 4 | 103 | Priya | DBMS |

## Step 2: Find students not enrolled in any course (LEFT JOIN).

```
SELECT s.student_id, s.student_name

FROM Students s

LEFT JOIN Enrollment e ON s.student_id = e.student_id

WHERE course_id IS NULL;
```

## Output

| | student_id [PK] integer | student_name character varying (100) |
|---|---|---|
| 1 | 104 | Amit |
| 2 | 105 | Neha |

### Step 3: Display all courses with or without enrolled students (RIGHT JOIN).

```
SELECT s.student_name, c.course_name
FROM students s
RIGHT JOIN enrollment e ON s.student_id = e.student_id
RIGHT JOIN courses c ON e.course_id = c.course_id;
```

## Output

| | student_name character varying (100) | course_name character varying (100) |
|---|---|---|
| 1 | Aradhya | DBMS |
| 2 | Aradhya | Operating Systems |
| 3 | Rohan | Thermodynamics |
| 4 | Priya | DBMS |

### Step 4: Show students with department info using MULTIPLE JOINS or SELF JOIN.

```
SELECT s.student_name, d.dept_name
FROM Students s
LEFT JOIN Departments d
   ON s.dept_id = d.dept_id
LEFT JOIN Enrollment e
   ON s.student_id = e.student_id;
```

## Output

| | student_name character varying (100) | dept_name character varying (100) |
|---|---|---|
| 1 | Aradhya | Computer Science |
| 2 | Aradhya | Computer Science |
| 3 | Rohan | Mechanical |
| 4 | Priya | Computer Science |
| 5 | Amit | Electrical |

## Step 5: Display all possible student-course combinations (CROSS JOIN).

```
SELECT s.student_name, c.course_name

FROM students s

CROSS JOIN courses c;
```

## Output

| | student_name character varying (100) | course_name character varying (100) |
|---|---|---|
| 1 | Aradhya | DBMS |
| 2 | Aradhya | Operating Systems |
| 3 | Aradhya | Thermodynamics |
| 4 | Aradhya | Networks |
| 5 | Rohan | DBMS |
| 6 | Rohan | Operating Systems |
| 7 | Rohan | Thermodynamics |
| 8 | Rohan | Networks |
| 9 | Priya | DBMS |
| 10 | Priya | Operating Systems |
| 11 | Priya | Thermodynamics |
| 12 | Priya | Networks |
| 13 | Amit | DBMS |
| 14 | Amit | Operating Systems |
| 15 | Amit | Thermodynamics |

## Learning Outcome

- **Join Understanding:** I learned how to use different types of SQL joins to retrieve related data from multiple tables.
- **Practical Database Skills**: I understood how real-world schemas such as Students, Courses, Enrollments, and Departments are related in a relational database.
- **Data Analysis Ability**: I learned how to identify matching and non-matching records using INNER JOIN, LEFT JOIN, and RIGHT JOIN**.**
- **Advanced SQL Usage**: I understood CROSS JOIN for generating all possible combinations**.**
- **PostgreSQL Practice:** I gained hands-on experience in writing and executing SQL queries in PostgreSQL / pgAdmin.